6710503739_HW#7_2

August 30, 2026

1 HW7 - Question 2

1.1 1. Load and prepare the data

[37]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, classification_report

df = pd.read_csv("../titanic_data.csv")
print(df.shape)
df.head()
```

```
(891, 12)
```

[37]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th…	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

[38]:
```
df.isna().sum()
```

[38]:
```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

[39]:
```
cols = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
X = df[cols].copy()
y = df["Survived"]

X["Age"] = X["Age"].fillna(X["Age"].median())
X["Embarked"] = X["Embarked"].fillna(X["Embarked"].mode()[0])

X = pd.get_dummies(X, columns=["Sex", "Embarked"], drop_first=True).astype(float)
X.head()
```

[39]:

	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q	Embarked_S
0	3.0	22.0	1.0	0.0	7.2500	1.0	0.0	1.0
1	1.0	38.0	1.0	0.0	71.2833	0.0	0.0	0.0
2	3.0	26.0	0.0	0.0	7.9250	0.0	0.0	1.0
3	1.0	35.0	1.0	0.0	53.1000	0.0	0.0	1.0
4	3.0	35.0	0.0	0.0	8.0500	1.0	0.0	1.0

[40]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)

print(X_train.shape, X_test.shape)
```

```
(712, 8) (179, 8)
```

1.2 2. KNN model

Parameters: n_neighbors=5 (odd no ties on a binary target), Euclidean distance on standardised features without it fare would dominate every distance.

[41]:
```
knn = KNeighborsClassifier(n_neighbors=5, metric="euclidean")
knn.fit(X_train_s, y_train)
knn_pred = knn.predict(X_test_s)
```

1.3 3. Logistic Regression model

Parameters: C=1.0 (default), solver='lbfgs', max_iter=1000 to guarantee convergence.

[42]:
```
logreg = LogisticRegression(C=1.0, solver="lbfgs", max_iter=1000, random_state=42)
logreg.fit(X_train_s, y_train)
log_pred = logreg.predict(X_test_s)
```

1.4 4. Compare the models

[43]:
```
print("KNN")
print(classification_report(y_test, knn_pred, target_names=["Died", "Survived"]))
print("Logistic Regression")
print(classification_report(y_test, log_pred, target_names=["Died", "Survived"]))
```

```
KNN
              precision    recall  f1-score   support

        Died       0.82      0.89      0.86       110
    Survived       0.80      0.70      0.74        69

    accuracy                           0.82       179
   macro avg       0.81      0.79      0.80       179
weighted avg       0.81      0.82      0.81       179

Logistic Regression
              precision    recall  f1-score   support

        Died       0.81      0.89      0.85       110
    Survived       0.79      0.67      0.72        69

    accuracy                           0.80       179
   macro avg       0.80      0.78      0.79       179
weighted avg       0.80      0.80      0.80       179
```

1.5 5. Explain the results

Findings. two models are separated by about one percentage - KNN reaches 0.816 accuracy / 0.744 F1 - Logistic Regression's 0.805 accuracy / 0.724 F1

Both models also fail in recall on Survived (0.70 and 0.67) is much lower than recall on Died (0.89 both). The classes are imbalanced (62% died) and the survivors who lack the strongest cues (female, 1st class) are the hard cases for either method.

The near-tie is itself informative: the Titanic signal is dominated by a few strong, roughly linear effects Sex, Pclass, Fare.

1.5.1 Advantages / disadvantages.

KNN `Adventages` - No training phase; captures non-linear, local patterns.

`Disadventage` - Sensitive to feature scaling and to irrelevant features.

Logistic Regression `Adventages` - stable, gives interpretable coefficients, odds ratios and calibrated probabilities.

`Disaventage` - Assumes a linear decision boundary